Dependency-Aware Thread Pool with Adaptive Task Scheduling — Implementation Plan

By Rewansh Choudhary · Operating Systems Lab Project

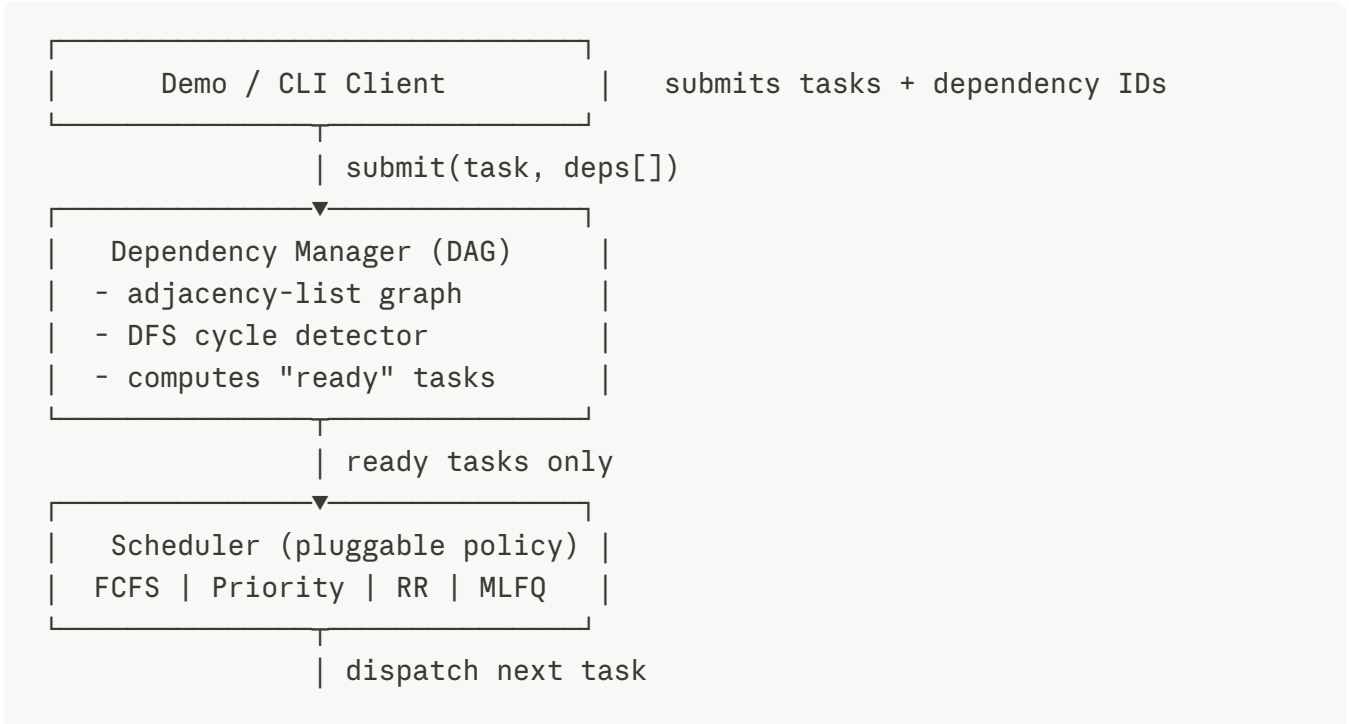
1. Project Overview

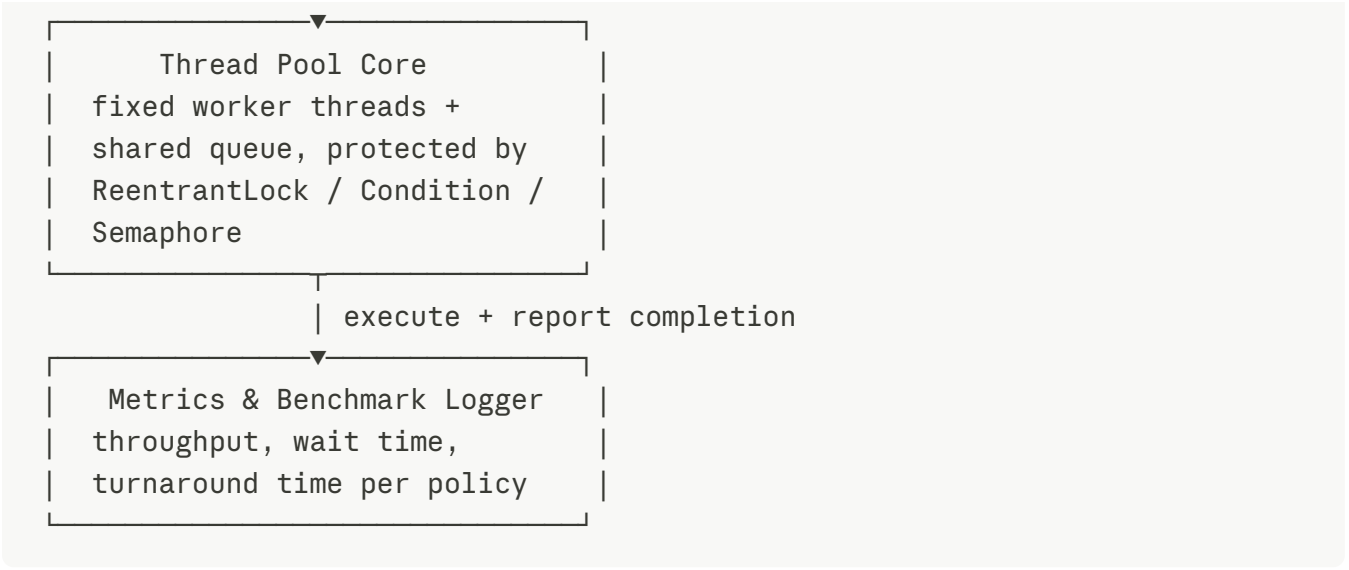
This project implements a custom, from-scratch thread pool in Java that goes beyond a standard fixed-worker-pool exercise in two ways: tasks can declare **dependencies** on other tasks (forming a Directed Acyclic Graph, or DAG), and the system performs **deadlock detection** on that dependency graph before execution begins. On top of this, the pool supports four scheduling policies — FCFS, Priority (with aging), Round Robin, and an adaptive Multilevel Feedback Queue (MLFQ) — and includes a benchmarking module to compare them empirically.

This document lays out the architecture, module breakdown, phased development plan, recommended dependencies, guidance on scoping the novel pieces sensibly, useful references, and a ready-to-use prompt for handing this spec to an AI coding agent.

2. Architecture

The system is organized into five layers, with data flowing top to bottom from task submission to execution:





The key architectural decision is that the **Dependency Manager sits above the Scheduler**: it never hands a task to the scheduler until all of that task's dependencies have completed. The scheduler itself has no knowledge of dependencies at all — it only ever sees a pool of "ready" tasks and decides which one runs next. This separation keeps each piece simple to implement, test, and explain individually.

3. Core Components

Component	Responsibility
`ThreadPoolManager`	Owns the fixed set of `WorkerThread`s; handles pool startup, graceful shutdown, and dynamic resizing based on queue depth
`WorkerThread` (extends `Thread`)	Loops: pull next ready task from the scheduler, execute it, report completion back to the Dependency Manager
`Task`	Holds id, priority, estimated burst time, list of dependency task-IDs, and current state (`PENDING` , `READY` , `RUNNING` , `DONE`)
`SchedulerPolicy` (interface)	Implemented by `FcfsScheduler` , `PriorityScheduler` (with aging), `RoundRobinScheduler` , `MlfqScheduler`
`DependencyGraph`	Adjacency-list representation; `addTask()` , `markComplete()` , `getReadyTasks()`

Component	Responsibility
`CycleDetector`	DFS with white/gray/black node marking; runs once at graph-build time to reject circular dependency sets before any task starts
`BenchmarkHarness`	Wraps a full run under one policy; records submit/start/finish timestamps per task; computes throughput, average wait time, turnaround time
`MetricsReporter`	Prints or exports a comparison table across all four policies

4. Development Plan (phased)

Phase 1 — target for Review 1

- Core thread pool: worker thread lifecycle, graceful shutdown
- Shared task queue synchronized with `ReentrantLock` + `Condition`
- FCFS policy, working end-to-end (submit → dispatch → execute)
- Baseline JUnit tests proving thread-safety: no lost tasks, no duplicate execution, no race conditions under concurrent submission

Phase 2 — target for Review 2

- Priority (with aging), Round Robin, and MLFQ policies
- Dependency graph with topological "ready" computation
- Cycle detector and the rejection path for invalid task sets
- Dynamic pool resizing based on queue depth
- Benchmark harness and the four-policy comparison report
- Final test suite, README, and a simple CLI demo client

5. Recommended Dependencies

- **Java 17+** — use records for the immutable `Task` metrics snapshot, and pattern matching (`switch` on sealed state) for clean state handling
- **Maven** (`pom.xml`) for build and dependency management
- **JUnit 5** (`org.junit.jupiter`) for unit and concurrency tests
- **SLF4J + Logback** (optional but recommended) — structured, thread-tagged logging is far easier to debug than scattered `println` calls when diagnosing interleaving bugs

- **No external graph library** — implement the DAG and cycle detector by hand with a plain adjacency list. A library like JGraphT would work, but hand-rolling it is small (under ~150 lines) and much easier to defend in a viva, since it directly demonstrates your own understanding of the algorithm
- **JMH** (Java Microbenchmark Harness) — optional, only worth adding if your evaluator specifically wants rigorous statistical benchmarking; a `System.nanoTime()`-based custom harness is sufficient and simpler to explain for a course project

6. Novel Idea Guidance (keeping it well-scoped)

The dependency-graph and deadlock-detection pieces are what make this project stand out, but they're also the easiest part to over-build. A few scoping choices that keep it demonstrable without turning into a research project:

- **Dependencies are explicit, not inferred.** A task lists the exact IDs it depends on at submission time. Don't try to auto-detect dependencies from shared data — that's a much bigger problem (static/dynamic data-flow analysis) and out of scope.
- **Cycle detection runs once, at graph-build time** — not repeatedly during scheduling. This keeps the algorithm simple (a single DFS pass) and keeps its cost out of the scheduling hot path.
- **Keep MLFQ to three priority levels.** Classic MLFQ designs can have many tiers with complex rules; three levels (high/medium/low) with a simple demotion-on-timeout and periodic priority boost is enough to demonstrate the concept clearly and is much easier to explain in a viva.
- **Design one clear demo scenario** rather than random generated graphs: e.g., a small "build pipeline" of 8–10 tasks representing something intuitive (compile → test → package → deploy style dependencies), with one or two tasks deliberately given a circular dependency so you can demonstrate the rejection path live. A relatable, visualizable scenario is more convincing to evaluators than an abstract stress test.
- **Resist adding distributed or persistence features.** Everything stays in-memory and single-JVM — that's already stated in your Scope section, and it's the right boundary for the timeframe you have.

7. Useful References

- *Operating Systems: Three Easy Pieces* (OSTEP) — free online textbook; the scheduling section's MLFQ chapter is the clearest available explanation of multilevel feedback queues and directly informs the demotion/promotion rules

- Silberschatz, Galvin & Gagne, *Operating System Concepts* — chapters on CPU Scheduling, Process Synchronization, and Deadlocks (for the detection-vs-avoidance-vs-prevention distinction)
- Oracle's official Java Concurrency tutorial (docs.oracle.com) — canonical usage patterns for `ReentrantLock`, `Condition`, and `Semaphore`
- CLRS (*Introduction to Algorithms*), Chapter 22 — the standard reference for DFS-based topological sort and cycle detection
- OpenJDK source for `java.util.concurrent` (e.g. `ThreadPoolExecutor`, `AbstractQueuedSynchronizer`) — useful to read for inspiration on real-world design patterns, but should not be copied; the point of this project is demonstrating the mechanics yourself
- Baeldung / GeeksforGeeks articles on "Producer-Consumer in Java" and "Topological Sort" — good as quick refreshers while implementing, not as a primary source

8. No-Code Prompt (for an AI coding agent)

The block below is self-contained and can be pasted directly into an AI coding assistant (e.g. Claude Code) to generate the implementation from scratch.

```
Build a Java 17+ Maven project called "dependency-aware-thread-pool" that
implements a custom thread pool with dependency-aware task scheduling. Do NOT
use java.util.concurrent.ExecutorService, Executors, or any pre-built thread
pool — the entire point is to implement thread management manually using
Thread, ReentrantLock, Condition, and Semaphore from
java.util.concurrent.locks and java.util.concurrent, so the low-level
mechanics are visible.

Requirements:

1. Task model: a Task has an id, priority (int), estimated burst time (ms), a
list of dependency task-IDs, and a state (PENDING, READY, RUNNING, DONE).

2. Thread pool core: a fixed-size pool of worker threads (extending Thread)
that pull the next ready task from a shared, lock-protected queue and execute
it. Use ReentrantLock + Condition for queue synchronization (no busy-
waiting). Support graceful shutdown. Support dynamic resizing of the worker
count based on queue depth (grow when queue depth exceeds a threshold, shrink
when idle).

3. Dependency graph: implement a DependencyGraph class using a plain
adjacency list (no external graph library). Tasks are submitted with their
dependency IDs. A task only becomes READY once all its dependencies report
DONE. Before accepting a batch of tasks, run a DFS-based cycle detector
```

(white/gray/black marking) over the dependency graph; if a cycle is found, reject the whole submission with a clear DeadlockDetectedException listing the cycle.

4. Scheduling policies: implement a SchedulerPolicy interface with four implementations: FcfsScheduler, PriorityScheduler (with priority aging so long-waiting low-priority tasks eventually get promoted), RoundRobinScheduler (fixed time quantum), and MlfqScheduler (3 priority levels, demote a task on timeout, periodically boost all tasks back to the top level to prevent starvation). The scheduler should be pluggable — the thread pool should work identically regardless of which policy is active.

5. Benchmarking: implement a BenchmarkHarness that runs an identical workload under each of the four policies and records submit time, dispatch time, and completion time per task, then computes throughput (tasks/sec), average waiting time, and average turnaround time. Output a comparison table across all four policies.

6. Demo: include a small CLI demo (Main class) that defines an 8-10 task "build pipeline"-style workload with realistic dependencies (e.g. compile -> test -> package -> deploy), runs it successfully, and then runs a second workload that deliberately contains a circular dependency to demonstrate the DeadlockDetectedException path.

7. Tests: write JUnit 5 tests covering (a) thread-safety of the core pool under concurrent submission (no lost or duplicated task execution), (b) correct dependency resolution ordering, (c) correct rejection of cyclic dependency graphs, and (d) correctness of each scheduling policy's ordering logic under a controlled task set.

8. Logging: use SLF4J + Logback with thread-name-tagged log output so execution interleaving is visible and debuggable.

9. Keep the implementation scoped to a single JVM, in-memory only — no networking, no persistence, no distributed coordination.

Deliver: full Maven project structure, all source files, the JUnit test suite, a README explaining how to build/run the demo and the benchmark, and a short results summary comparing the four scheduling policies on the sample workload.

9. Extension Idea: Comparing Against Java Virtual Threads

Java Virtual Threads (finalized as a standard feature in JDK 21 under JEP 444) are a genuinely strong fit for this project's OS theme — not just a trendy library to name-drop. Unlike the platform threads this project builds its pool from, virtual threads are **not** backed one-to-one by an OS kernel thread. Instead, the JVM multiplexes many thousands of virtual threads onto a small pool of OS "carrier" threads, scheduling and parking them in user space. This is a direct, real-world instance of the **many-to-many (M:N) threading model** and the **user-level vs. kernel-level threads** distinction from OS coursework, so it connects cleanly to material you're already covering rather than being an unrelated add-on.

Suggested extension (keep it secondary, not a replacement): add an optional second execution mode to the benchmark harness that runs the identical workload using `Thread.ofVirtual().start(...)` instead of the custom platform-thread pool. Use an I/O-bound synthetic workload (tasks that call `Thread.sleep()` to simulate blocking I/O) rather than a CPU-bound one — this is where the difference is dramatic and easy to explain: the custom platform-thread pool needs one OS thread per concurrently blocked task, while virtual threads release their carrier thread while parked, so thousands of blocked tasks cost almost nothing. Report this as a third comparison column (platform-thread custom pool vs. the four scheduling policies vs. virtual threads) in the same benchmark table from Section 4.

Important scoping note: this should stay clearly labeled as a reference comparison, not folded into the core deliverable. The entire point of the hand-built thread pool is to demonstrate manual, OS-level thread management yourself — the same reason `ExecutorService` was ruled out earlier in this plan. Virtual threads abstract that mechanics away by design, so they're valuable here as a *contrast* that shows you understand both models, not as a substitute implementation.

Version note: this extension requires **JDK 21+**, one step above the JDK 17+ baseline used for the core project. If you add this, bump the project's JDK version accordingly (JDK 21 and 24 are both current LTS releases, so either works).

Reference: JEP 444 ("Virtual Threads"), openjdk.org — the authoritative source for the design rationale and the M:N scheduling model.

If using the No-Code Prompt in Section 8, you can hand the agent this optional add-on separately once the core is done:

(Optional extension) Add a third execution mode to the BenchmarkHarness that runs the same I/O-bound workload using Thread.ofVirtual().start() (JDK 21+) instead of the custom pool, and add its throughput/wait-time/turnaround results as an extra column in the comparison report. Clearly label this mode as a reference comparison against the custom implementation, not a replacement for it.